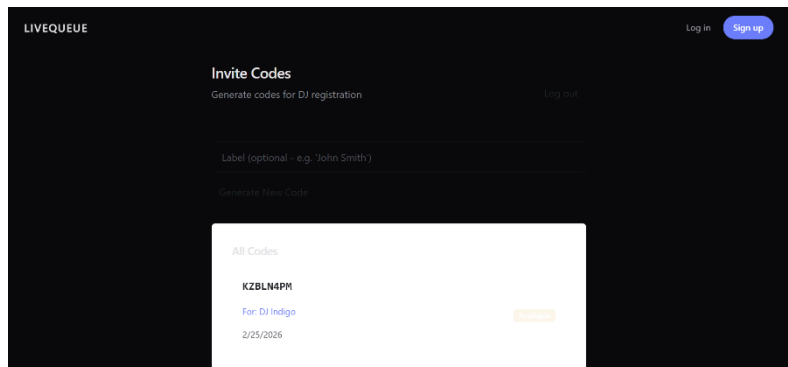


Anvesh Sapkota | Product Case Study: *LiveQueue* ([Link](#))

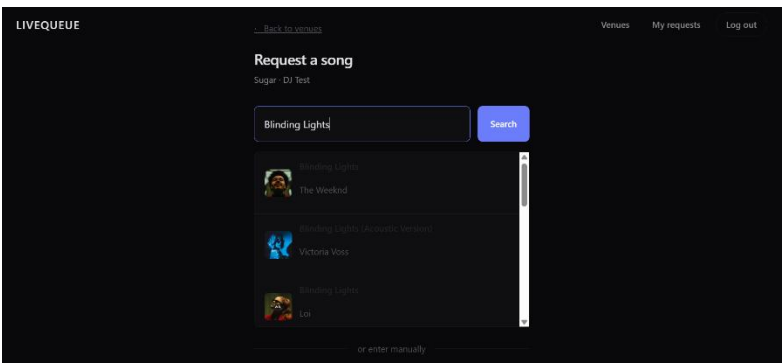
Executive Summary

- **The Problem:** Independent live performers operate as small businesses but frequently miss out on organic monetization during live sets. Audience requests are currently handled through chaotic, high-friction interactions (shouting, handing over cash) that drain the performer’s cognitive load and result in lost revenue.
- **The Solution:** *LiveQueue* is a real-time fan-creator engagement platform that acts as a digital point-of-sale and workflow manager for song requests. It structures audience demand into a manageable queue, unlocking new revenue streams while giving performers total control.
- **Key Product Insights:**
 - **Creator Monetization:** Designed the platform to treat performers as independent businesses, equipping them with tools for dynamic pricing and structured queue prioritization.
 - **Transaction Workflow:** Architected a future payment flow utilizing authorization holds instead of immediate payment capture to prevent refund margin erosion and protect unit economics.
 - **AI-Accelerated Strategy & Build:** Leveraged AI tools (Claude, Cursor) as a strategic sparring partner to debate platform constraints, model pricing structures, scope the market, and rapidly iterate on the technical prototype.

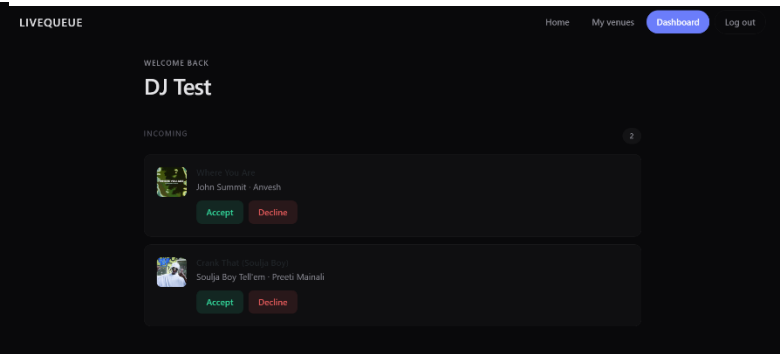
Visuals + Process



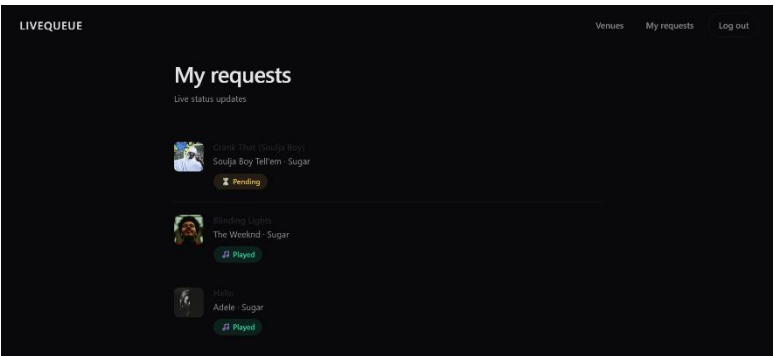
The admin-only invite code generator acts as a strict quality gate for the marketplace's supply side



A streamlined request experience designed to capture audience intent quickly, optimized for rapid adoption in the current free MVP



Optimized for high-energy environments, the dashboard uses high-contrast elements to help performers make split-second decisions



Reduces cognitive load by organizing requests into clear, actionable states: Pending, Accepted, Played, or Declined.

Part 1: Product Strategy & Execution

Insight: Software in live, high-energy environments should support human judgment, not replace it.

Product Principles

- **Performer Control is Non-Negotiable:** Every request requires performer approval, preserving creative ownership.
- **Structure Reduces Cognitive Load:** Requests are converted into a visible queue with clear states (Pending → Accepted → Played / Declined), removing ambiguity for attendees and simplifying decision-making for performers.
- **Accessibility Drives Liquidity:** A configurable free-request mechanism lowers the barrier to entry while allowing paid prioritization to coexist, balancing inclusivity with monetization.

Designing for a Multi-Sided System: LiveQueue operates across three distinct participant groups with competing incentives:

- **Attendees (Buyers):** Seek responsiveness, influence, and a seamless payment experience.
- **Performers (Sellers):** Require autonomy, monetization pathways, and minimal operational distraction.
- **Venues (Ecosystem):** Prioritize guest experience, extended dwell times, and operational flow.

The Payment Flow: To prevent refund mechanics from quietly eroding margins, I identified authorization holds as a superior alternative to immediate payment capture:

1. Place a temporary hold when a request is submitted.
2. Capture funds *only* if the performer accepts the request.
3. Release funds instantly if declined.

This approach will protect unit economics, eliminate costly chargebacks, and maintain a frictionless user experience once monetization is activated.

Success Metrics

- **Marketplace Health:** Performer acceptance rate, decision latency, requests per hour.
- **Adoption & Liquidity:** Requests per attendee, Free → Paid conversion rate.
- **Experience Quality:** Repeat usage, fulfillment rate.

Technical Trade-offs & Execution

- **De-risking Through Constraints (Deezer vs. Spotify):** Originally scoped for the Spotify API, I pivoted the MVP to the Deezer Simple API. It required no OAuth complexity or API keys, allowing for immediate prototype validation and faster iteration without sacrificing necessary track metadata.

- **Supply-Side Quality Control:** To prevent platform dilution, I explicitly rejected open DJ registration. Instead, I architected an admin-only invite code system to serve as a strict quality gate for the supply side of the marketplace.
- **Design as a Trust Signal:** Replaced standard template gradients with a monochromatic, high-contrast interface. For a product acting as a digital point-of-sale, visual polish directly impacts perceived credibility while reducing cognitive load in dark, live environments.

Part 2: AI-Native Decision Making

LiveQueue was shaped through an AI-native decision-making process. I treated AI as a structured thinking partner rather than an answer engine, using it to interrogate ideas and pressure-test system behavior before committing to implementation.

Problem Discovery & Reframing: Through iterative dialogue with Claude, the central challenge evolved from a feature question (“*How do we let people request songs?*”) to a behavioral one (“*How do we capture audience intent without turning performers into jukeboxes?*”).

Marketplace Simulation & Assumptions I deliberately debated AI recommendations to evaluate their robustness in a multi-sided marketplace, specifically exploring:

- **Pricing Models:** Evaluating how dynamic pricing influences user behavior and platform liquidity.
- **Incentive Alignment:** Simulating fairness between free and paid participation.
- **Risk Mitigation:** Uncovering the risks of over-automation and the necessity of performer veto power.

Product Feature Negotiation with AI: I used AI (Cursor/Claude) not just to generate code, but to negotiate feature scope. For example, when building the DJ invite system, the AI suggested building a complex relational database validation to track who received which code. I pushed back, recognizing that operational flexibility was more important than database normalization for an MVP. We aligned on a simple, optional text label that allows admins to track codes with human-readable tags, saving hours of development time while still solving the user need.

Impact on the Build Process: Using AI-assisted tools significantly shortened the exploration cycle. It allowed me to evaluate multiple solution paths rapidly and focus on high-quality product decisions rather than prematurely committing to a technical direction.

Part 3: Current State & Future Roadmap (v2)

As a solo builder, I developed the current iteration of *LiveQueue* as a functional, free-tier prototype. This allowed me to rapidly build the core queueing mechanics and validate the interface design without the overhead of immediate financial compliance.

With the core technical architecture and UX established, the future roadmap focuses on commercialization and real-world deployment:

- **Integrating the Payment Gateway:** Implementing the authorization hold workflow defined above to manage paid requests and protect margins.
- **Dynamic Pricing Engine:** Building the logic for performers to set minimum bid thresholds based on real-time venue capacity and queue length.
- **Live Beta Testing:** Piloting the system at local campus venues to validate assumptions around performer cognitive load and attendee adoption in a live, high-energy setting.